LLM-based Agents: Tutorial

Autonomous Software Agents

A.A. 2025-2026

Dr. Marco Bombieri



UNIVERSITY OF TRENTO - Italy
Department of Information
and Communication Technology

LLM vs LLM agents

LLM

- *Responds to a prompt by generating text*
- Produces explanations, suggestions, or information

LLM Agent

- *Acts in the external world to achieve a goal specified in the prompt*
- It can plan, use tools (APIs, databases, software), and execute actions

---

Prompt: *"Organize a trip to Rome for tomorrow with a budget of €150"*

- Suggests possible trains
- Recommends budget hotels
- Proposes a travel itinerary

→ **Provides information**

- Searches train availability via APIs
- Compares hotel prices
- Books the train and hotel
- Sends a confirmation email

→ **Performs actions to achieve the goal**

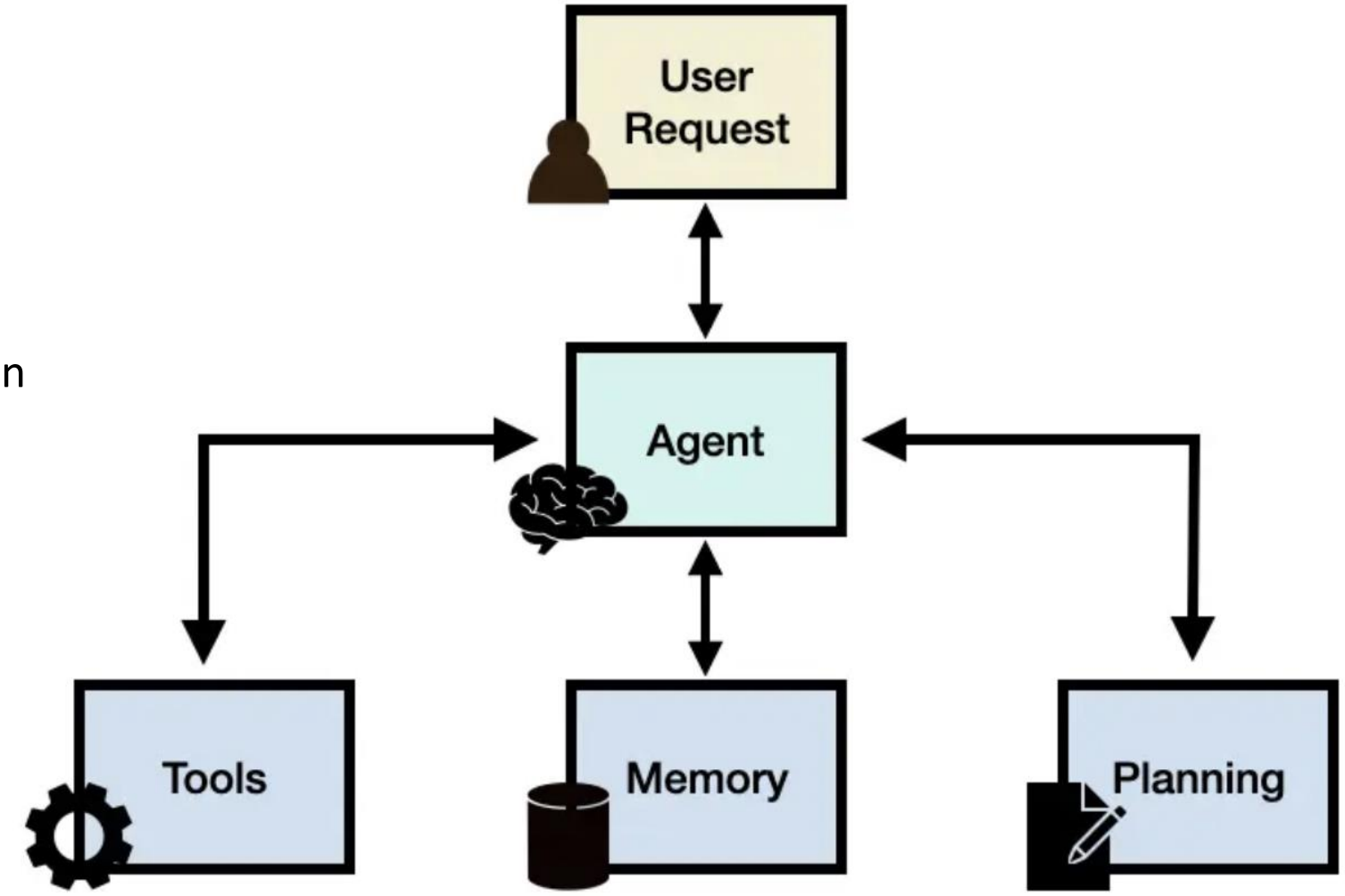
“LLM responds.
LLM Agent acts”

What are LLM agents?

- LLM-based agents (LLM agents) are **AI applications** that execute complex tasks using LLMs combined with **modules** like **planning**, **memory**, and **tool usage**
- **Core components**
  - **LLM as the Controller** – Acts as the 'brain' to guide operations
  - **Planning Module** – Breaks down tasks into structured steps
  - **Memory Module** – Stores and retrieves past interactions
  - **Tool Usage Module** – Interacts with external tools and APIs

How LLM Agents Work

- The LLM processes user input and determines the required steps
- The agent uses memory to maintain context
- Planning ensures systematic execution of tasks
- The agent leverages external tools when necessary



LLM Laboratory agenda

- **First LLM Call**
  Connect to an LLM through an OpenAI-compatible API and send a simple prompt.
- **Chat Loop and Conversation Memory**
  Build a terminal chat and understand how conversation history is stored and reused.
- **Manual Tool Use**
  Add a deterministic tool and call it manually from the chat.
- **Model-Selected Actions**
  Let the LLM decide when a tool should be used.
- **Action → Tool → Observation → Final Answer**
  Build the first complete agentic loop.
- **Tool Registry**
  Generalize tool execution so new tools can be added easily.
- **Execution Loop**
  Allow the agent to perform multiple tool calls before answering.
- **Planner-Guided Execution**
  Add a planner that creates a sequence of steps.
- **DeliverooJS Integration**
  Connect the agent to an environment where tools can read and change state.
- **Exercises**

Some exercises useful to understand better how an agenticAI system is desinged

What are we building?

A minimal LLM-based agent:

- it will receive a user request, decide what needs to be done, use external tools when necessary, and produce a final answer.
- We will start with a simple LLM call.
- Then, step by step, we will add the missing components until it becomes a prototype of an LLM-based agent.
- At the end we will integrate it with DeliverooJS Game

LLM calls

Step 1: simple LLM call - overview

Goal: Implement a one-shot LLM call

You need:

- A .env file:

LITELLM_BASE_URL=**https://llm.bears.disi.unitn.it/v1** → Proxy LiteLLM that contain the LLMS

LITELLM_API_KEY=<YOUR-API-KEY> → The API-KEY needed to access the BEARS' models

LOCAL_MODEL=llama-3.3-70b-lmstudio → Name of the LLM we use

Download the file: 01_llm-simple-user-prompt.mjs

- It loads the configuration from .env
- It configures the LLM endpoint and create an OpenAI-compatible client
- It send the first prompt to the LLM and print the answer

Install the dependencies and run the script:

npm install openai dotenv duck-duck-scrape

node 01_LLM-simple-call.mjs

Step 1: simple LLM call – configuration

- Download the file 01_llm-simple-user-prompt.mjs :

```
// LOAD the configuration from .env file and configure the LLM endpoint
import "dotenv/config";
……
const baseURL = process.env.LITELLM_BASE_URL || "https://llm.bears.disi.unitn.it/v1";
const apiKey = process.env.LITELLM_API_KEY;
const MODEL = process.env.LOCAL_MODEL || "llama-3.3-70b-lmstudio";

// Create an OpenAI-compatible client
import OpenAI from "openai";
…
const client = new OpenAI({
    baseURL,
    apiKey,
});
```

Step 1: simple LLM call – the prompt

```
const response = await client.chat.completions.create({
    model: MODEL,
    // messages: the conversation we are sending to the model
    messages: [
                {
                  role: "user",
                  content: "Explain the difference between an LLM and an LLM-based agentic AI system",
                },
    ],
    // The level of determinism (creativity): 0.0 -> determinism (no creativity)
    temperature: 0.1,
    });
```

It creates the message specifying:

- **model**
- **messages array**
- **temperature**

```
console.log(response.choices[0].message.content);
```

It prints the answer

Step 1: – messages and temperature

- The **messages** array is the input conversation for the LLMs. Each element has a **role** and some **content.**
- **role** tells the model who is speaking:
  - **“user” → the user request**
  - **“system” → instructions about how the model should behave**
  - **“assistant” → previous model responses**
- **content** contains the message.
- You can add more **role/content** elements to create a **conversation memory**

- **temperature** controls the variability of the model’s output.
  - A low value, such as 0.0, makes the response more stable and predictable;
  - a higher value makes the response more creative but less deterministic.

Step 1: exercise

- Download the file 01_llm-simple-user-prompt.mjs
- Create the .env file as explained before
- Install the dependences and execute the file.
- Modify the **messages** array so that it includes both:
  - A **system message** : the LLMs must answer with a maximum of 20 words!
  - a **user message** that contains the actual question.

- Modify the user message with the **content**: “How much is 377834873478 * 974829994?” Check the results with a calculator. Is the answer correct? Why?

Step 2: Chat Loop with Conversation Memory

Goal: Transform the first one-shot LLM call into a simple terminal chatbot and introduce the first form of memory.

What changes from the previous step?

1. **More inputs**: the user can type multiple messages
2. **Conversational memory**: previous user and assistant messages are stored in the **messages** array and reused.
3. **Chat Loop**: the program keeps running and conversational memory is saved until the user types "**exit**"

Step 2: 1

Download the file “02_chat_simple.mjs”

1. Import terminal input utilities

```
import readline from "node:readline/promises";

import { stdin as input, stdout as output } from "node:process";
```

2. Initialize conversation memory

```
const messages = [
        {
            role: "system",
            content: "You are a concise teaching assistant.",
        },
];
```

Step 2: 2

3. Start a terminal chat loop and store the user message

```
const rl = readline.createInterface({ input, output });
. . .
while (true) {
    const userInput = await rl.question("You: ");
    if (userInput.trim().toLowerCase() === "exit") {
            break;
    }
 . . .
    messages.push({
        role: "user",
        content: userInput,});
```

It creates a command-line interface

It creates the chat loop that ends by typing “exit”

The user message is stored in the “messages” array

Step 2: 3

4. Send the full conversation to the model

```
const response = await client.chat.completions.create({
                model: MODEL,
                messages,
                temperature: 0.1,
});

const assistantMessage = response.choices?.[0]?.message?.content ?? "";
. . .
```



We are not sending only the latest message but the full messages array

5. Add also the assistant's answer to the conversation history

```
messages.push({
                role: "assistant",
                content: assistantMessage,
});
```



The assistant's answer is now in the messages array: the next model call can see it

Step 2: Chat Loop with Conversation Memory

The model does not automatically remember the previous conversation. It remembers only because our program sends the previous messages again at every turn.

This approach does not scale well. If the conversation becomes very long, the messages array keeps growing:

- Turn 1 → short messages array
- Turn 10 → larger messages array
- Turn N → conversation too large for the model's context window → out of memory error!

In real application, your implementation should decide what to keep in the context window and forget (or save externally, or summarize) less important information.

Step 2: exercise

- Download and execute the file 02_chat-simple.mjs
- Create a file “02_chat-complete” with the following implementation:
- If the user types “/memory”, the conversation memory is printed
- If the user types “/reset”, the conversation memory is cleared
- If the user types “/save”, the conversation memory is saved in a file

At this stage, the user (not the model) decides what to keep and what to forget. In real applications, the program itself should be designed to manage memory automatically!

Tools

Step 3: Introducing tools

Goal: Add the first external tool to our chat program

So far, the model could only generate text; with step 3 the program can also execute a deterministic tool: this is our first step toward tool-using agents

What changes from the previous step?

We add:

1. A tool: calculator function;
2. A manual (for the moment!) command to call it; If the user starts the message with \calc, the tool is called
3. a way to store the tool result inside the conversation memory.

At this stage, the model does **not** decide when to use the tool. → This is not an agent yet!

Step 3: Introducing tools

Download the file “03_tool-manual.mjs”

1. **Define the tool**

```
function calculate(expression) {
    try {
            return String(eval(expression));
    } catch (error) {
            return `Error: ${error.message}`;
    }
}
```



The tool here is a simple function. It could also be an API call

This function receives a mathematical expression as a string and returns the result.

Step 3: Introducing tools

2. Tool usage and result storage

```
if (command.startsWith("/calc ")) {
    const expression = userInput.slice("/calc ".length).trim();
    const result = calculate(expression);
```

Inside the chat loop, we check whether the user typed a calculator command

```
messages.push({
  role: "user",
  content: `Tool observation: calculate(${expression}) = ${result}`,
});
```

Store the tool result as an observation for the conversation memory

The tool result is not automatically known by the model. The model can use its result later only because we inserted it into the messages array.

Step 3: Exercise

Download the file “03_tool-manual.mjs”

1. Try this to check if the tool is used and if the LLM remembers the result:

   You: /calc 12 * 7

   …

   You: What was the result of the calculation?

2. Try this and check the results with a calculator. Is the answer correct? Why?

   /calc 377834873478 * 974829994?

Please note: for now, the user decides when to call the tool; with step 4, we will let the model decide when a tool should be used

Please note: return String(eval(expression)); is not safe for production: tool inputs should be validated and restricted before execution!

Step 4: model selected actions

Goal: Let the LLM decide when a tool should be used

Previously, the user had to write: /calc 12*7

With step 4 the user can ask: How much is 12*7? ; the model should decide that the calculator tool is needed

What changes from the previous step?

Download the file 04-automatic-tool-selection.mjs:

1. The calculator tool is still the same
2. The terminal chat loop is almost unchanged BUT….
3. **We must change the system prompt**! → Prompt Engineering is a crucial step!

Given a prompt, e.g., How much is 12*7?

The LLM should produce an action in the format:

Action: TOOL NAME ← The function (tool) name

Action input: 3 * 12 ← The parameters of the function

Step 4: Weak Prompt VS Strong Prompt

USER: Can you help me calculate the price if 3 tickets cost 17 euros each?

```
const messages = [
                {
                role: "system",
                content: "You are a concise teaching assistant." +
                                "You can use a calculator when needed.",
                },
];
```

Example of a **weak prompt**: this is ambiguous and the model may simply answer "The total price is 51 euros".
But we want to produce an action in the format:
Action: calculate
Action Input: 3 * 17

```
const messages = [
                {
                role: "system",
                content: ` You are an AI assistant. Available tools:
                - calculate(expression): evaluates a mathematical expression.
                If the user asks for something that requires calculation, do not
compute it yourself. Instead, respond exactly in this format:

                Action: calculate
                Action Input: <mathematical expression>

                If no tool is needed, answer normally.
                },
];
```

This prompt **works better** because:
- It explains to the LLM when it is necessary to use a tool and when it is not.
- It explains to the LLM the format of the response they should return when they intend to use a tool.

Expected model output:
Action: calculate
Action Input: 3 * 17

Step 4: Why prompt Engineering Matters

- In this step, the main change is not in the JavaScript logic.
- The main change is in the **system prompt: b**y changing the prompt, we change the role of the LLM:

  Before: The model answers the user directly.

  After: The model decides whether a tool is needed and describes the action to perform.

- The prompt defines the contract between the model and the program, i.e., between language reasoning and executable actions:

  What tools exist? When should they be used? What output format should the model follow? What should the model avoid doing?

Step 4: How to design a good prompt?

- In complex applications, prompt engineering can become a very time-consuming part of the development process.
- There is no predefined recipe that always produces a good prompt.
- A prompt that works well for one model, one task, or one tool may fail in a slightly different situation.
- Iterative process:

  write an initial prompt
  ↓
  test it on concrete examples
  ↓
  observe where the model fails
  ↓
  make the instructions more precise
  ↓
  test again

You can also provide to LLMs some example of the desired output
↓
Few-Shot Learning

Step 4: What we have now

- At this point, the tool is **not executed automatically yet**.
- The model is only producing text that represents an action: this is the first step toward an agentic system.
- The model starts translating natural language into an action and it is not calling the JavaScript function directly: it is only writing a structured message that our program will later parse it!

Step 5: Exercise: complete the first agentic Loop

- At this point, the tool is **not executed automatically yet**.
- Download the file 05-TODO_automatic-tool-selection-execution.mjs
- Complete the missing TODO sections with the instruction provided in the code.

GOAL of the exercise is to implement the following pipeline:

User request → LLM decides whether a tool is needed ← **Already implemented!**

→ Program parses the action

→ Program executes the tool

→ Tool returns an observation

→ LLM produces the final answer (this requires a new prompt and a new LLM call)

← **New**

Step 6: more tools

Goal: Adding more tools

So far, our agent can use only one tool: calculate(expression)

In the step 6, we introduce a second tool: get_current_time(location) that returns the current local time.

What changes from the previous step?

1. We introduce a second tool
2. We modify the prompt to handle the second tool too
3. We introduce a **tool registry**

Step 6: What is a tool registry?

Let's suppose to have several tools:

```
function getCurrentTime(location) {
  . . . .                    . . .
}
```

```
function calculate(expression) {
  . . . .
}
```

With only one tool we can write:

```
if (action === "calculate") {
            observation = calculate(actionInput);
```

But this does not scale: if we add more tools, we do not want a long chain of if statements

Instead, we introduce a **tool registry:**

```
const TOOLS = {
            calculate,
            get_current_time: getCurrentTime,
            // add new tools here
};
```

Tool registry is an object that maps the tool name used by the model to the actual function implemented in the program. With a tool registry, adding a new tool is easy

With tool registry now we can simply write: TOOLS[actions](actionInput) instead of if statements

Step 6: Exercise

Download the file 06_TODO_more_tools.mjs

Replace the TODO in the code with your implementation to integrate the second tool

In particular:

1. Update the ACTION_PROMPT so that the model knows the new tool exists. It is a good idea to provide an example of the desired output
2. Add a tool registry: replace the hardcoded tool execution logic with a registry and execute tools through it.
3. Execute the code: is it robust?
4. Try the prompt "Calculate 12 * 7 and then 5 + 3" . Does your agent call calculate(expression) twice? Why?

Planners

Step 7: Execution loops

Goal: Adding an execution loop

Until now, the agent could perform only one tool call before producing a final answer. That worked for simple requests such as:

How much is 12 * 7? OR What time is it in Rome? This is not enough for request that require multiple steps: What time is it in Rome, and how much is 12 * 7?

To answer correctly, the agent needs to:

1. call get_current_time("Rome")
2. observe the result
3. call calculate("12 * 7")
4. observe the result
5. produce the final answer

What we have

User request
↓
LLM chooses one action
↓
Tool execution
↓
Observation
↓
Final answer

What we need

User request
↓
LLM chooses an action
↓
Tool execution
↓
Observation
↓
LLM chooses next action or final answer
↓
...

Step 7: the new implementation

Download the file "07A_execution_loop.mjs"

Previously, the program had a fixed structure:

- one LLM decision
- one optional tool call
- one final answer

Now, the program runs an agent loop:

```
for (let i = 0; i < maxIterations; i++) {
            . . .
}
```

At each iteration, the model can choose one of two options:

Action: <tool name>
Action Input: <tool input>
OR
Provide an answer without using a tool

The execution loop must have a limit; without it the agent could get stuck forever

We asked the model to do so, with prompt engineering

Step 7: the agent loop in code

`runAgentTurn(...)` handles one complete user request. Inside the loop, the model is called repeatedly until either:

- The model produces a final answer
- The maximum number of iterations is reached

```
async function runAgentTurn(userInput, maxIterations = 6) {
   messages.push({
      role: "user",
      content: userInput,
   });

   for (let i = 0; i < maxIterations; i++) {
      . . .
   }
}
```

Step 7: One iteration of the loop

```
async function runAgentTurn(userInput, maxIterations = 6) {
  . . .
  for (let i = 0; i < maxIterations; i++) {
    . . .
  }
}
```

At each iteration, the program asks the model what to do next:

```
const assistantMessage = await callModel(messages, { temperature: 0 });
```

Then it stores the model output:

```
messages.push({
  role: "assistant",
  content: assistantMessage,
});
```

The output may contain either:

- **an action**
- **a Final Answer**

Step 7: if the model requests a tool

The program first checks whether the model produced an action:

```
const parsedAction = extractAction(assistantMessage);
```

If so, the program executes the corresponding tool:

```
if (TOOLS[action]) {
        observation = await TOOLS[action](actionInput);
}
```

And the observation is added to memory:

```
messages.push({
        role: "user",
        content: `Observation: ${observation}`,
});
```

The loop continues, so the model can use the observation to decide the next step.

Step 7: if the model produces a final answer

If no tool is needed, the program checks for a final answer:

```
const finalAnswer = extractFinalAnswer(assistantMessage);
```

If the final answer exists, the turn is complete:

```
if (finalAnswer) {
        console.log(`Assistant: ${finalAnswer}\n`);
}
```

This is the stopping condition of the loop.

Step 7: Adding a “Thought Step”

We also modified the prompt to ask the model to include a short reasoning step before deciding what to do:

. . .
If you need to use a tool, output ONLY:

Thought: <brief reasoning>
Action: <tool name>
Action Input: <tool input>
. . .

Expected output:
Thought: I need to calculate the expression.
Action: calculate
Action Input: 12 * 7

This is useful because we can see why the model selected a tool.

Step 7: Exercise 1

In 07A_execution_loop.mjs, the global messages array stores everything:

- User request
- Assistant action
- Tool observation
- Assistant action
- Tool observation
- Final answer

This is useful for learning because we can inspect the full trace with /memory, but it also means that the visible conversation memory is mixed with the agent's internal scratchpad.

Modify the code 07A-execution_loop.mjs to separate conversation memory from internal agent scratchpad (reasoning).

This separation is closer to how real agent systems are usually designed.

Step 7: Exercise 2

Modify the previous code so that the runtime checks the model output before trusting it.
The runtime should detect:

1. More than one Action in the same model output
2. Action and Final Answer in the same model output

And it should enforce this rule:

- If an Action is present, execute the Action first.
- Accept a Final Answer only when there is no Action in the output.

Example:

Thought: I need the current time.
Action: get_current_time
Action Input: Rome
Final Answer: The current time in Rome is 12:00.



This is dangerous because the model has included a final answer before the tool has actually been executed

Step 8: From Execution Loop to Planned Agent

Goal: Adding a planner

Until now, the agent decided what to do step by step during the execution loop. Now we introduce a clearer architecture:

Planner → Step Executor → Final Answer Builder

The agent first creates a plan, then executes each planned step explicitly, and finally builds the answer for the use.

Before:

The LLM controlled the sequence dynamically

Now:

The JavaScript runtime controls the sequence of planned steps.

Depending on the type of application you may prefer this approach or the previous one: both are correct and used in the practice.

Step 8: New architecture

Download the code 08B-planner-execution-loop.mjs

This code separates the agent into three modules.

1. Planner
   Creates a short list of concrete steps.
2. Step Executor
   Executes exactly one step at a time.
3. Final Answer Builder
   Combines all step results into the final response.



Each module has its own prompt and its own responsibility.

Step 8: The Planner

Download the code 08B-planner-execution-loop.mjs

```
const PLANNER_PROMPT = `

You are a planning module inside an AI agent. Your job is to break the user's request into a short sequence of concrete steps.
. . .
Return exactly this JSON shape:

{
        "steps": [
        "step 1",
        "step 2"
        ]
}
```

The planner must produce structured output so the program can easily parse it.

If the model returns valid JSON, the program can extract it and iterate over the steps:

parsedPlan.steps

The Step Executor receives the current step, the full plan, and completed step results so far and it can either call a tool or complete the step.

Step 8: Runtime controls the plan

The JavaScript program explicitly loops over the plan steps. The executor is no longer free to decide the next high-level step. It must execute the current step.

```
const completedResults = [];

for (const step of plan.steps) {
   const execution = await executeStep(step, {
   userInput,
   plan,
   completedResults,
});

completedResults.push(execution.result);
}
```

After the full plan has been executed, the final answer is generated:

```
const finalAnswer = await buildFinalAnswer(userInput, plan,completedResults);
```

Step 8: Summary

- Planner:

What should be done?

- Executor:

How do I complete this specific step?

- Final Answer Builder:

How do I communicate the result to the user?

Each component is still powered by the same LLM. The difference is the prompt and the role assigned to the model.

LLM-based Agents in DeliverooJS

Step 9: Integration with DeliverooJS

Goal: Integration with DeliverooJS

Until now, the agent could use tools that only returned information:

- calculate(expression)
- get_current_time(location)

With this step we connect the agent to an external environment: **DeliverooJS**

With step 9 our agent can interact with DeliverooJS using also the tools:

- get_my_position()
- move(direction)

With step 9 the agent can:

- Read the state of an external environment
- Change the state of the environment

Step 9: modify the .env and install dependencies

- Modify your .env by adding this information:

DELIVEROOJS_URL=https://deliveroojs.bears.disi.unitn.it ← **NEW!**

DELIVEROOJS_TOKEN=<YOUR-DELIVEROOJS-TOKEN> ← **NEW!**

LITELLM_BASE_URL=https://llm.bears.disi.unitn.it/v1

LITELLM_API_KEY=<YOUR-LITELLM_API_KEY>

LOCAL_MODEL=llama-3.3-70b-lmstudio

- Install the @unitn-asa/deliveroo-js-sdk dependency:

npm install @unitn-asa/deliveroo-js-sdk

- Download the file 9_07C_DeliverooAgent.mjs

- Run it with: node 9_07C_DeliverooAgent.mjs

Step 9: Connecting to DeliverooJS

- To connect to DeliverooJS, we add one input:

```
import { DjsConnect } from "@unitn-asa/deliveroo-js-sdk/client";
```

- We create the socket connection that gives the program access to the DeliverooJS env:

```
const deliverooUrl = process.env.DELIVEROOJS_URL;
const deliverooToken = process.env.DELIVEROOJS_TOKEN;
const socket = DjsConnect(deliverooUrl, deliverooToken);
```

Step 9: Tracking the Agent State

- We keep a local copy of the agent state:

```
const me = {
    id: null,
    name: null,
    x: null,
    y: null,
    score: 0,
};
```

- Then we update it whenever DeliverooJS sends new information:

```
socket.onYou((you) => {
    me.id = you.id;
    me.name = you.name;
    me.x = you.x;
    me.y = you.y;
    me.score = you.score;
});
```

Step 9: new Tool: get_my_position

- We implement a tool that reads the current local state of the agent.

```
async function getMyPosition() {
   console.log("---- GET MY POSITION ----");
   if (me.x === null || me.y === null) {
      return "Error: agent position is not available yet.";
   }
   return JSON.stringify({
      id: me.id,
      name: me.name,
      x: me.x,
      y: me.y,
      score: me.score,
      });
}
```

Step 9: new Tool: move

- We implement a tool that changes the state of the environment.

```
async function move(direction) {
    const normalized = direction.trim().toLowerCase();
    const validDirections = ["up", "down", "left", "right"];
    try {
            const result = await socket.emitMove(normalized);
            if (result) {
                    return `Successfully moved ${normalized}. New position:
            ${JSON.stringify(result)}.`;
    }
    return `Error: failed to move ${normalized}.`;
    } catch (error) {
            return `Error: moving ${normalized} failed: ${error.message}`;
    }
}
```

Step 9: update Tool Registry

The tool registry now includes both the previous tools and the DeliverooJS tools:

```
const TOOLS = {
    calculate,
    get_current_time: getCurrentTime,
    get_my_position: getMyPosition,
   move,
};
```

Thanks to the tool registry, the runtime still uses the same generic execution logic!

```
observation = await TOOLS[action](actionInput);
```

Step 9: update prompt

The prompt must tell the model about the new tools and their rules:

Available tools:

- calculate(expression): evaluates a mathematical expression
- get_current_time(location): returns the current local time for Rome/Roma
- get_my_position(): returns the agent's current x, y coordinates and score
- move(direction): moves the agent one step in one direction: up, down, left, or right

. . .

- move(up) increases y by 1
- move(down) decreases y by 1
- move(right) increases x by 1
- move(left) decreases x by 1

. . .

The agentic architecture remains the same!

Step 9: Exercise

Download the file 9_07C_DeliverooAgent.mjs

1. Run the model with the following prompts; does it work as expected?
   - Move right twice and then tell me your final position.
   - What is your position? Go in x+2 and y-3
   - Extract the hours from the actual time in Rome, then subtract 10 to it and move up of the result

2. You may need to reset the memory after each prompt with /reset . Can you modify your agent to do it automatically when needed?

3. At the moment your agent receives the prompt from your command line. Can you modify it so that it receives the prompt from the DeliverooJS chat?

Step 10: Exercise

- Modify the code to introduce the following tools:

getTemperature(location) → Given a location, it returns the actual location's temperature

HINT:

You can use the API geocoding-api.open-meteo.com/v1 to extract the latitude and longitude of the location and use the API api.open-meteo.com/v1 to extract the weather of the location in the corresponding latitude and longitude

webSearch(URL, query) → it finds the answer to a "query" on the webpage at the URL

- Modify the code to introduce useful tools for a more complete integration with the DeliverooJS game

- Then, modify the tool registry and the prompt to include also these tools.

Thank you for your attention!